

The Infer static analyser – An Industrial Application

Infer is a static analysis tool; given some source code it produces a list of potential bugs. Thus, it intercepts critical bugs before they have shipped to users, and helps prevent crashes or poor performance.

Infer is **part of Continuous Integration** cycle of Java, Java for Android, C, C++, and Objective C at Meta. It is being used by AdaCore, AmazonWebServices, Facebook, Freefem, Instagram, Microsoft, mozilla, oculus, sonatype, spotify, Tangram, Uberm WhatsApp, CodeAI, SD.com, Marks and Spencer, Netcetera, OLA, Sky, Tile, Vuo, wolfSSL. Infer does **not produce false positives**.

It is based on Separation Logic and Incorrectness Logic. Both these logics are based on Hoare Logics. The development of these logics started in 2000, by Peter O' Hearn and John Reynolds. Philippa Gardner and Azalea Raad, and many more from Imperial College, have made major contributions.

More at <https://fbinfer.com>.

Slide 1

1 Operational Semantics for While

The language *While* of simple *while programs* has a grammar consisting of three syntactic categories: *numeric expressions*, which represent natural numbers; *booleans*, which are similar to expressions but represent truth values rather than numbers; and *commands*, which are imperative statements which affect the store of the computer.

Slide 2

Syntax of While

$$B \in \text{Bool} ::= \text{true} \mid \text{false} \mid E =_s E \mid E <_s E \\ \mid B \&_s B \mid \neg_s B$$

$$E \in \text{Exp} ::= x \mid n \mid E +_s E$$

$$C \in \text{Com} ::= x := E \mid \text{if } B \text{ then } C \text{ else } C \\ \mid C; C \mid \text{skip} \mid \text{while } B \text{ do } C$$

$x \in \text{Var}$ ranges over the variable identifiers. $n \in \mathbb{N}$ ranges over the natural numbers. We use brackets where necessary to disambiguate.

The commands C are:

- the *noop* command, `skip`, which does nothing;
- the *assignment* command, `$x := E$` , which updates the variable x with the value obtained by evaluating the expression E ;
- the *conditional* command, `$\text{if } B \text{ then } C_1 \text{ else } C_2$` , which runs the command C_1 if the Boolean condition B evaluates to true, or C_2 if B evaluates to false;
- *sequential composition*, `$C_1; C_2$` , which first runs the command C_1 and then runs the command C_2 (note that the semicolon is an operator joining two commands into one, and not just a piece of punctuation at the end of a command);
- the *loop constructor*, `$\text{while } B \text{ do } C$` , which, if the Boolean condition B evaluates to true, runs the command C and returns to the start of the loop, and, if B evaluates to false, does nothing.

This informal English specification gives you some idea of what the commands do. We will study a formal mathematical definition of the behaviour of *While* programs by defining an operational semantics. We first study the operational semantics of simple expressions (without variables), since it is

helpful to introduce the basic concepts of the operational semantics using such simple terms.

Slide 3

Questions

- Why write $=_s$ rather than $=$?
- Why $<_s$, $\&_s$, $\neg_s$, and $+_s$, rather than $<$, $\&$, $\neg$, and $+$?
- Are the following terms in Com . If yes, then show how, and if not, then say why not.

(A) $\text{if } x =_s 3 +_s y \text{ then skip else } z := 17$

(B) $x := 3 +_s y;$

(C) $x := 3 +_s y; y := 33$

(D) $\text{if } x =_s 3 +_s y \text{ then skip else } z := 17; u := 6$

(E) $\text{if } x =_s 3 +_s y \text{ then skip else } (z := 17; u := 6)$

- To distinguish the syntactic expression constructor $+_s$ from mathematical addition $+$ on numbers, we will write addition on numbers as $+$ unless there is no chance of ambiguity. While we consider $2 + 1$ to be mathematically equal to 3, the expressions $2 +_s 1$ and 3 are syntactically different.
- Similarly for $=_s$, $<_s$, $\&_s$, and $\neg_s$ in boolean expressions.
- We apply the definition from the previous slide:
 - (A) (1) $x, 3, y \in Exp$, therefore, $3 +_s y \in Exp$ (by rules for Exp . Therefore $x =_s 3 +_s y \in Bool$.
 - (2) $skip \in Com$.
 - (3) $17 \in Exp$, therefore $x := 17 \in Com$.
 - (4) From (1), (2) and (3) we obtain $\text{if } x =_s 3 +_s y \text{ then skip else } z := 17 \in Com$.
 - (B) Not in Com , because $;$ is a separator, not a terminator.
 - (C) (1) $3, y \in Exp$, therefore $3 +_s y \in Exp$.

- (2) $x := 3 +_s y \in Com$, by (1) and rules for Com .
 - (3) $33 \in Exp$, therefore $y := 33 \in Com$.
 - (4) $x := 3 +_s y; y := 33 \in Com$, by (2), (3) and rules for Com .
- (D) The term is syntactically well-formed, but ambiguous. Namely, it can be derived as
- (if $x =_s 3 +_s y$ then skip else $z := 17$); $u := 6$)
- or as
- if $x =_s 3 +_s y$ then skip else ($z := 17; u := 6$)
- For the former, we use
- (1) $\text{if } x =_s 3 +_s y \text{ then skip else } z := 17 \in Com$, as already demonstrated in (A).4.
 - (2) $u := 17 \in Exp$, using similar argument to that in (C).3.
 - (3) $(\text{if } x =_s 3 +_s y \text{ then skip else } z := 17); u := 6 \in Exp$ by (1) and (2) and rules for Com .
- (E) Similar argument as in (D).

As we said in the earlier slide, we use brackets where necessary disambiguate.

1.1 Operational Semantics for *SimpleExp*

It is quite straightforward to change the interpretation of expression n to range over the integers ... $-2, -1, 0, 1, 2, \dots$

An operational semantics for *SimpleExp*

We define a smaller expression language, *simpleExp*, for simple expressions without variables.

$$E \in \text{SimpleExp} ::= n \mid E +_s E$$

Slide 4

An operational semantics for *SimpleExp* will tell us how to evaluate an expression to get a result. This can be done in two ways:

- *small-step, or structural, operational semantics* gives a method for evaluating an expression step-by-step;
- *big-step, or natural, operational semantics* ignores the intermediate steps and gives the result immediately.

1.1.1 Inference Rules

Aside: Inference Rules

Inference rules can describe term execution, or type systems, or other, inductively defined entities. They have one of the two forms below:

$$\frac{A_1 \ A_2 \dots \ A_n}{B} \qquad \overline{C}$$

The left rule defines that if $A_1 \wedge A_2 \dots \wedge A_n$ then B . The right rule defines that C always holds. We sometimes write them in the form

$$(\text{name-of-rule}) \frac{A_1 \ A_2 \dots \ A_n}{B} \text{side-cond}$$

where A_1, A_2, A_n , and B are assertions about the same, inductively defined entity (a set or relation or function), and *side-cond* is a further condition, *not* about that entity. The meaning is that $A_1 \wedge A_2 \dots \wedge A_n \wedge \text{side-cond}$ implies B

Note: While the slides and sample answers place the side- condition next to the line, it is OK if the students write any side-conditions above the line.

An *inductively-defined* is used to define the elements in a set in terms of other elements in the set. Some examples of recursively-definable objects include factorials, natural numbers, Fibonacci numbers, etc. An inductive definition of a function defines values of the function for some inputs in terms of the values of the same function for other (usually smaller) inputs. For example, the factorial function $n!$ is defined by the rules

$$0! = 1 \quad (n+1)! = (n+1) \cdot n!.$$

1.1.2 Big-step semantics for SimpleExp

Let us consider big-step semantics first. The big-step semantics for SimpleExp takes the form of a relation $\Downarrow$ between expressions and values, which are those expressions we deem to be a 'final answer'. For expressions, we take values to be the natural numbers, writing

$$E \Downarrow n.$$

Slide 5

$\Downarrow$ Big-step Semantics of *SimpleExp*

$$\Downarrow \subseteq \text{SimpleExp} \times \mathbb{N}$$

$$(B\text{-NUM}) \frac{}{n \Downarrow n}$$

$$(B\text{-ADD}) \frac{E_1 \Downarrow n_1 \quad E_2 \Downarrow n_2}{E_1 +_s E_2 \Downarrow n_3} \quad n_3 = n_1 + n_2$$

We can give similar rules for multiplication and other natural operations.

Slide 6

Intuitively, a rule such as

$$\frac{E_1 \Downarrow n_1 \quad E_2 \Downarrow n_2}{E_3 \Downarrow n_3}$$

means that, if it is the case that $E_1 \Downarrow n_1$ and also $E_2 \Downarrow n_2$, then it is the case that $E_3 \Downarrow n_3$. We call $E_1 \Downarrow n_1$ and also $E_2 \Downarrow n_2$ the premises of the rule, and $E_3 \Downarrow n_3$ the conclusion. When there are no entries above the line, the rule is an axiom, which is to say that it always holds.

The rules define a relation $\Downarrow$ which says when an expression evaluates to a final answer. We say that $E \Downarrow n$ is in this relation, or $E \Downarrow n$ holds, only if it can be established from the axioms and rules. So, if we want to assert that, for example, $3 +_s (2 +_s 1) \Downarrow 6$, we need to show this to be the case by applying the axioms and rules given in the definition of $\Downarrow$.

Questions

Slide 7

- *Is the rule (B-NUM) superfluous?*
- *Find a number n such that $3 +_s (2 +_s 1) \Downarrow n$, and prove that assertion.*

A Proof that $3 +_s (2 +_s 1) \Downarrow 6$

Slide 8

$$\begin{array}{c}
 \begin{array}{ccc}
 & (B\text{-NUM}) \frac{}{2 \Downarrow 2} & (B\text{-NUM}) \frac{}{1 \Downarrow 1} \\
 (B\text{-ADD}) \frac{(B\text{-NUM}) \frac{}{3 \Downarrow 3} \quad (B\text{-ADD}) \frac{2 \Downarrow 2 \quad 1 \Downarrow 1}{2 +_s 1 \Downarrow 3}}{3 +_s (2 +_s 1) \Downarrow 6}
 \end{array}
 \end{array}$$

*This is the **derivation tree** of $3 +_s (2 +_s 1) \Downarrow 6$.
As you see, the derivation tree requires the rule (B-NUM) at the leaves.*

There are two natural properties of the big-step semantics for *SimpleExp*: *determinacy*, which says that an expression can evaluate to at most one answer; and *totality*, which says that an expression must evaluate to at least one answer.

Some properties of $\Downarrow$

Determinacy

$$\forall E \in \text{SimpleExpr}. \forall n_1, n_2 \in \mathbb{N}. \\ [E \Downarrow n_1 \wedge E \Downarrow n_2 \implies n_1 = n_2].$$

Totality

$$\forall E \in \text{SimpleExpr}. \exists n \in \mathbb{N}. E \Downarrow n.$$

Slide 9

1.1.3 Small-step Semantics of *SimpleExp*

The big-step semantics given above tells us what the final value of an expression is straight away. The rules tell us how to compute the answer, but sometimes it is desirable to be more explicit about exactly how programs are evaluated. A small-step semantics lets us do just this. We shall define a relation

$$E \rightarrow E'$$

which describes one step of evaluation of E .

Small-step Semantics of *SimpleExp*

$$\rightarrow \subseteq \text{SimpleExpr} \times \text{SimpleExpr}$$

Slide 10

$$(S\text{-LEFT}) \quad \frac{E_1 \rightarrow E'_1}{E_1 +_s E_2 \rightarrow E'_1 +_s E_2}$$

$$(S\text{-RIGHT}) \quad \frac{E \rightarrow E'}{n +_s E \rightarrow n +_s E'}$$

$$(S\text{-ADD}) \quad \frac{}{n_1 +_s n_2 \rightarrow n_3} \quad n_3 = n_1 + n_2$$

Note the difference between the signature of $\Downarrow$ and of $\rightarrow$

These rules say: to evaluate an addition, first evaluate the left-hand argument; when you get to a number, evaluate the right-hand argument; when you get to a number there too, add the two together to get a number. Note that there are no rules to evaluate a number, because it has already been fully evaluated.

Consider the expression $3 +_s (2 +_s 1)$. By the axiom (S-ADD), we have $2 +_s 1 \rightarrow 3$ so, by rule (S-RIGHT), we have $3 +_s (2 +_s 1) \rightarrow 3 +_s 3$. The axiom also says that $3 +_s 3 \rightarrow 6$, so we have

$$3 +_s (2 +_s 1) \rightarrow 3 +_s 3 \rightarrow 6.$$

It is important to realise that the order of evaluation is fixed by this semantics. We have

$$(1 +_s 2) +_s (3 +_s 4) \rightarrow 3 +_s (3 +_s 4),$$

not $(1 +_s 2) +_s 7$. The big-step semantics cannot make such a fine-grained stipulation.

1.1.4 Getting the final answer

While the intermediate expressions of a computation are interesting, we are ultimately concerned with the final answer yielded by evaluating an expression. To capture this mathematically from the small-step semantics, we define the relation which expresses multiple-step evaluations.

Many Steps of Evaluation

Given a relation $\rightarrow$, we define a new relation $\rightarrow^$ by:*

$E \rightarrow^ E'$ holds if and only if either $E = E'$ (so no steps of evaluation are needed to get from E to E') or there is a finite sequence*

$$E \rightarrow E_1 \rightarrow E_2 \dots \rightarrow E_k \rightarrow E'.$$

This relation $\rightarrow^$ is called the reflexive transitive closure of $\rightarrow$.*

For our expressions, we say that number n is the final answer of E if $E \rightarrow^ n$.*

Slide 11

Slide 12

Normal Form

An expression E is in *normal form* (and said to be *irreducible*), if there is no E' such that $E \rightarrow E'$.

Theorem

The normal forms of expressions are the numbers.

Analogous to the big-step semantics, determinacy for the small-step semantics means that each expression can only evaluate (for one step) in at most one way. Note that we do not have totality, since normal forms do not evaluate any further. A more general property of $\rightarrow$ is confluence, which states that if we choose two different evaluation paths for an expression then they can both be extended so that they eventually converge. Since $\rightarrow$ is deterministic, confluence follows trivially, but we shall see other semantics where determinacy does not hold, but confluence does. Normalisation for $\rightarrow$ means that every evaluation path must eventually reach a normal form; it cannot go on reducing forever. (There is also a property called weak normalisation, which means that every expression has some evaluation path that eventually reaches a normal form. This is less interesting for our semantics here, but is interesting when we consider the lambda calculus later in the course.) Since $\rightarrow$ is both confluent and normalising, it follows that every expression evaluates to exactly one normal form. As we know, the normal forms are the numbers. These properties are typically proved by induction, as we shall see. Weak normalization means that there exist at least one evaluation path which eventually reaches a normal form. Strong normalization means that every evaluation path eventually reaches a normal form.

Some Properties of $\rightarrow$

Determinism

For all E, E_1, E_2 , if $E \rightarrow E_1$ and $E \rightarrow E_2$, then $E_1 = E_2$.

Confluence For all E, E_1, E_2 , if $E \rightarrow^* E_1$ and $E \rightarrow^* E_2$, then there exists E' such that $E_1 \rightarrow^* E'$ and $E_2 \rightarrow^* E'$

Weak Normalisation For any expression E_1 , there exists a finite sequence of expressions $E_2, \dots, E_k$ such that, E_k is in normal form, and for all $i \in [1..k)$, $E_i \rightarrow E_{i+1}$.

Strong Normalisation There are no infinite sequences of expressions $E_1, E_2, E_3, \dots$ such that, for all i , $E_i \rightarrow E_{i+1}$.

Unique Normal Form For all E, n_1, n_2 , if $E \rightarrow^* n_1$ and $E \rightarrow^* n_2$ then $n_1 = n_2$.

Slide 13

The relation $\rightarrow$ for expressions as defined in the previous slides has all the properties defined here. It is deterministic, confluent, weakly normalizing, strongly normalizing, and has unique normal forms.

However, these properties do not hold for all execution relations. Imagine, eg that we added to the definition of $\rightarrow$ the inference rule

$$\frac{}{0 +_s E \rightarrow E}$$

Then, the new relation $\rightarrow$ would no longer be deterministic, but it would be confluent.

Or, imagine that we added the rule

$$\frac{}{E_1 +_s E_2 \rightarrow E_2 +_s E_1}$$

With this addition, $\rightarrow$ would not be deterministic but could be confluent, it would have unique normal form, and would be weakly normalizing, but not strongly normalizing.

1.2 Generalization: Properties of rewrite relations $\rightsquigarrow$

The discussion in the previous subsection demonstrates that it makes sense to study the properties of such binary, execution (or rewrite) relations in general, and to study whether some properties are stronger than other properties.

Generalization: Properties of rewrite relations $\rightsquigarrow$

Assume a set A , and a binary relation $\rightsquigarrow \subseteq A \times A$. Then we define

For $a, a' \in A$ we say $a \rightsquigarrow^* a'$ if $a = a'$ or $\exists a'' \in A. [a \rightsquigarrow a'' \wedge a'' \rightsquigarrow^* a']$

$a \in A$ is in **normal form** if $\neg [\exists a' \in A. a \rightsquigarrow a']$

$\rightsquigarrow$ is **deterministic** if $\forall a, a_1, a_2 \in A. [a \rightsquigarrow a_1 \wedge a \rightsquigarrow a_2 \Rightarrow a_1 = a_2]$.

$\rightsquigarrow$ is **confluent** if $\forall a, a_1, a_2 \in A. [a \rightsquigarrow^* a_1 \wedge a \rightsquigarrow^* a_2 \Rightarrow \exists a' \in A. [a_1 \rightsquigarrow^* a' \wedge a_2 \rightsquigarrow^* a']]$

$\rightsquigarrow$ is **weakly normalizing** if $\forall a \in A. \exists d \in A. [a \rightsquigarrow^* d \wedge d \text{ is in normal form}]$

$\rightsquigarrow$ is **strongly normalizing** if $\forall a \in A. \text{there is no infinite sequence } a_1, \dots, a_n, \dots \text{ such that } \forall i \in \mathbb{N}. a_i \rightsquigarrow a_{i+1}$.

$\rightsquigarrow$ is **has unique normal forms** if $\forall a, a_1, a_2 \in A. [a \rightsquigarrow^* a_1 \wedge a \rightsquigarrow^* a_2 \wedge a_1, a_2 \text{ are both in normal form} \Rightarrow a_1 = a_2]$.

Slide 14

Note that the definition of $\rightsquigarrow^*$ is inductive.

Any deterministic relation is confluent, but not the other way round.

Any strongly normalizing relation is weakly normalizing, but not the other way round.

Any deterministic relation has unique normal forms.

1.3 Relating $\Downarrow$ and $\rightarrow$

Slide 15

Relating $\Downarrow$ and $\rightarrow^*$ **Theorem**

$$\forall E \in \text{SimpleExpr}. \forall n \in \mathbb{N}. [E \Downarrow n \iff E \rightarrow^* n].$$

Both $\Downarrow$ and $\rightarrow$ give semantics to expressions in terms of the numbers to which they evaluate. Although they were defined differently, we can prove a theorem which shows that they agree.

1.4 Operational Semantics of *While*

We give a small-step operational semantics to the programming language *While*, whose syntax was introduced in slide 1 and is repeated in slide 12. The first immediate issue is what it means to evaluate an assignment:

$$x := E \rightarrow ?$$

where E might now contain variables. We need more information about the state of the machine's memory. Slide 13 gives the definition of a state suitable for modelling *While*. Intuitively, a state s tells us what, if anything, is stored in the memory location corresponding to each variable of the language, and $s[x \mapsto n]$ is the state s updated so that the location corresponding to x contains n . For example, consider the state $s[x \mapsto 4][y \mapsto 5][z \mapsto 6]$. We have

$$s[x \mapsto 4][y \mapsto 5][z \mapsto 6](y) = 5$$

$$s[x \mapsto 4][y \mapsto 5][z \mapsto 6](z) = 6$$

$$s[x \mapsto 4][y \mapsto 5][z \mapsto 6](x) = 4$$

Our small-step semantics will therefore be concerned with programs together with their state.

Syntax of *While*

$$B \in \text{Bool} ::= \text{true} \mid \text{false} \mid E =_s E \mid E <_s E \mid \dots \\ \mid B \&_s B \mid \neg_s B \mid \dots$$

$$E \in \text{Exp} ::= x \mid n \mid E +_s E \mid \dots$$

$$C \in \text{Com} ::= x := E \mid \text{if } B \text{ then } C \text{ else } C \\ \mid C; C \mid \text{skip} \mid \text{while } B \text{ do } C$$

where x is variable and n is a number.

States

A **state** is a partial function from variables to numbers
($State = Var \rightarrow \mathbb{N}$) such that $s(x)$ is defined for finitely many x .

E.g.,

$$s_1 = (x \mapsto 4, y \mapsto 5, z \mapsto 6)$$

describes a partial function where variable x has value 4, y has value 5, and z has value 6.

Our small-step semantics for While will be defined using **configurations** of the form $\langle E, s \rangle$, $\langle B, s \rangle$ and $\langle C, s \rangle$. This expresses that we evaluate E , B and C with respect to state s , and may modify s in the process.

Slide 17

State Update

We define the state update operation $[v \mapsto n]$ on states as follows

$$\begin{aligned} s[v \mapsto n](u) &= n && \text{if } u = v \\ &= s(u) && \text{otherwise} \end{aligned}$$

where v and u are variables, and n is a number.

For example, with

$$s_1 = (x \mapsto 4, y \mapsto 5, z \mapsto 6)$$

we have that

$$s_1[y \mapsto 88] = (x \mapsto 4, y \mapsto 88, z \mapsto 6)$$

and

$$s_1[w \mapsto 7] = (x \mapsto 4, y \mapsto 5, z \mapsto 6, w \mapsto 7)$$

Slide 18

1.4.1 Expressions and Booleans

Expressions and booleans do not present any difficulty. The only new kind of expression is the variable. Its semantics involves fetching the appropriate value of the variable from the state (the variable store). The expression evaluation relation $\rightarrow_e$ is described using the rules in slide 14. The rules describing the boolean evaluation relation $\rightarrow_b$ are left as an exercise. We usually omit the subscripts e and b , since it is clear from the context which evaluation relation is appropriate. We use them in the formal definitions, to clarify the interactions between these relations and also the execution relation $\rightarrow_c$ for our language While.

Expressions

$$\rightarrow_e \subseteq (\text{Exp} \times \text{State}) \times (\text{Exp} \times \text{State})$$

$$(W\text{-EXP.LEFT}) \quad \frac{\langle E_1, s \rangle \rightarrow_e \langle E'_1, s' \rangle}{\langle E_1 +_s E_2, s \rangle \rightarrow_e \langle E'_1 +_s E_2, s' \rangle}$$

$$(W\text{-EXP.RIGHT}) \quad \frac{\langle E, s \rangle \rightarrow_e \langle E', s' \rangle}{\langle n +_s E, s \rangle \rightarrow_e \langle n +_s E', s' \rangle}$$

$$(W\text{-EXP.VAR}) \quad \frac{}{\langle x, s \rangle \rightarrow_e \langle n, s \rangle} \quad s(x) = n$$

$$(W\text{-EXP.ADD}) \quad \frac{}{\langle n_1 +_s n_2, s \rangle \rightarrow_e \langle n_3, s \rangle} \quad n_3 = n_1 + n_2$$

Slide 19

Exercise: Now write down the rules for booleans.

Notes: The rules for evaluating expressions do not affect the state. Can you explain in words why this is the case? How might you prove it (difficult at the moment)?

1.4.2 Commands

The command execution relation has the form $\langle C, s \rangle \rightarrow_c \langle C', s' \rangle$. The rules for $\rightarrow_c$ (again, we often omit the subscript c) are different from the rules for $\rightarrow_e$ and $\rightarrow_b$, in that they will directly alter the state. Intuitively, we want our rules to show how the commands update the state, and we will know that a command has finished its work when it reduces to *skip*. We shall now consider each command in turn and write down the appropriate rules. For assignment, $x := E$, we first want to evaluate E to some number n , and then update the state so that x contains the number n . For sequential composition, $C_1; C_2$, we first allow C_1 to run to completion, changing the state as it does so, and then compute C_2 from the resulting state. For conditionals, we first evaluate the boolean guard: if it returns *true* we take the first branch; if it returns *false*, we take the second branch.

Commands

$$\rightarrow_c \subseteq (Com \times State) \times (Com \times State)$$

Assignment:

$$(W\text{-ASS.EXP}) \quad \frac{\langle E, s \rangle \rightarrow_e \langle E', s' \rangle}{\langle x := E, s \rangle \rightarrow_c \langle x := E', s' \rangle}$$

$$(W\text{-ASS.NUM}) \quad \frac{}{\langle x := n, s \rangle \rightarrow_c \langle skip, s[x \mapsto n] \rangle}$$

Slide 20

Assignment – Example

Consider the state $s_1 = (x \mapsto 4, y \mapsto 5, z \mapsto 6)$ from earlier.

Then, execution of $z := y +_s 7$ proceeds as follows

$$\begin{aligned} \langle z := y +_s 7, s_1 \rangle &\rightarrow_c \langle z := 5 +_s 7, s_1 \rangle \rightarrow_c \langle z := 13, s_1 \rangle \\ &\rightarrow_c \langle skip, s_2 \rangle \end{aligned}$$

where $s_2 = (x \mapsto 4, y \mapsto 5, z \mapsto 13)$

And execution of $v := y +_s 7$ proceeds as follows

$$\begin{aligned} \langle v := y +_s 7, s_1 \rangle &\rightarrow_c \langle v := 5 +_s 7, s_1 \rangle \rightarrow_c \langle v := 13, s_1 \rangle \\ &\rightarrow_c \langle skip, s_3 \rangle \end{aligned}$$

where $s_3 = (x \mapsto 4, y \mapsto 5, z \mapsto 6, v \mapsto 13)$

Slide 21

Commands - continued

Sequential Composition

Slide 22

$$(W\text{-SEQ.LEFT}) \quad \frac{\langle C_1, s \rangle \rightarrow_c \langle C'_1, s' \rangle}{\langle C_1; C_2, s \rangle \rightarrow_c \langle C'_1; C_2, s' \rangle}$$

$$(W\text{-SEQ.SKIP}) \quad \frac{}{\langle \text{skip}; C_2, s \rangle \rightarrow_c \langle C_2, s \rangle}$$

Commands – continued

Conditional

Slide 23

$$(W\text{-COND.TRUE}) \quad \frac{}{\langle \text{if true then } C_1 \text{ else } C_2, s \rangle \rightarrow_c \langle C_1, s \rangle}$$

$$(W\text{-COND.FALSE}) \quad \frac{}{\langle \text{if false then } C_1 \text{ else } C_2, s \rangle \rightarrow_c \langle C_2, s \rangle}$$

(W-COND.BEXP)

$$\langle B, s \rangle \rightarrow_b \langle B', s' \rangle$$

$$\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow_c \langle \text{if } B' \text{ then } C_1 \text{ else } C_2, s' \rangle$$

What about *while*? Intuitively, we want to evaluate the boolean guard, and, if true, run the command then go back to the beginning and start again. Consider as a first try slide 18. The problem with this approach is that the only rule we've got which is capable of entering the loop body is the one for *while true do C*, which ought to be an infinite loop. By evaluating the boolean guard in place, as in the first rule of slide 18, we have made a serious error. The point is that we do not want to evaluate that boolean once and use that value for ever more, but rather we want to evaluate that boolean every time we go through the loop. So, when we evaluate it the first time, it is vital that we don't throw away the 'old' *B*, which this rule does. The solution is to make a copy of *B* to evaluate each time. In fact, we are able to give a single rule for *while* using the conditional command, as shown in slide 19.

Incorrect Semantics for *while*

$$(W\text{-}WHILE?) \frac{\langle B, s \rangle \rightarrow_b \langle B', s' \rangle}{\langle \text{while } B \text{ do } C, s \rangle \rightarrow_c \langle \text{while } B' \text{ do } C, s' \rangle}$$

$$(W\text{-}WHILE?) \frac{}{\langle \text{while } false \text{ do } C, s \rangle \rightarrow_c \langle skip, s \rangle}$$

$$(W\text{-}WHILE?) \frac{}{\langle \text{while } true \text{ do } C, s \rangle \rightarrow_c \langle ? \rangle}$$

Slide 24

Correct Semantics for while

Slide 25

$$(W\text{-}WHILE) \frac{}{\langle \text{while } B \text{ do } C, s \rangle \rightarrow_c \langle \text{if } B \text{ then } (C; \text{while } B \text{ do } C) \text{ else skip}, s \rangle}$$

All this rule does is ‘unfold’ the while loop once. If we could write down the infinite unfolding, there would be no need for the `while` syntax.

1.4.3 An Example

Slide 20 shows a program for computing the factorial of x and storing the answer in variable a . Let s be the state $(x \mapsto 3, y \mapsto 2, a \mapsto 9)$, using an obvious notation for states. It should be the case that

$$\langle C, s \rangle \rightarrow^* \langle \text{skip}, s' \rangle$$

where $s'(a) = 6$. (Can you predict the final values of x and y ?) Let's check that this is correct. First some abbreviations: we write C' for the sub-program

```
while  $y >_s 0$  do
  ( $a := a \times_s y$ ;
    $y := y -_s 1$ )
```

and $s_{i,j,k}$ for the state $(x \mapsto i, y \mapsto j, a \mapsto k)$. Thus, for our factorial example, the initial state s can be written $s_{3,2,9}$. Now let's do the evaluation. Each line should really be justified by reference to one of the rules of the operational semantics.

A factorial program
$$\begin{aligned} C = & \ y := x; \\ & \ a := 1; \\ & \ \textit{while } y >_s 0 \textit{ do} \\ & \quad (\ a := a \times_s y; \\ & \quad \quad y := y -_s 1 \) \end{aligned}$$
Slide 26


```

    ⟨y := x; a := 1; C', s3,2,9⟩
→ ⟨y := 3; a := 1; C', s3,2,9⟩
→ ⟨skip; a := 1; C', s3,3,9⟩
→ ⟨a := 1; C', s3,3,9⟩
→ ⟨skip; C', s3,3,1⟩
→ ⟨C', s3,3,1⟩
→ ⟨if y >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,3,1⟩
→ ⟨if 3 >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,3,1⟩
→ ⟨if true then (a := a ×s y; y := y −s 1; C') else skip, s3,3,1⟩
→ ⟨a := a ×s y; y := y −s 1; C', s3,3,1⟩
→ ⟨a := 1 ×s y; y := y −s 1; C', s3,3,1⟩
→ ⟨a := 1 ×s 3; y := y −s 1; C', s3,3,1⟩
→ ⟨a := 3; y := y −s 1; C', s3,3,1⟩
→ ⟨skip; y := y −s 1; C', s3,3,3⟩
→ ⟨y := y −s 1; C', s3,3,3⟩
→ ⟨y := 3 −s 1; C', s3,3,3⟩
→ ⟨y := 2; C', s3,3,3⟩
→ ⟨skip; C', s3,2,3⟩
→ ⟨C', s3,2,3⟩
→ ⟨if y >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,2,3⟩
→ ⟨if 2 >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,2,3⟩
→ ⟨if true then (a := a ×s y; y := y −s 1; C') else skip, s3,2,3⟩
→ ⟨a := a ×s y; y := y −s 1; C', s3,2,3⟩
→ ⟨a := 3 ×s y; y := y −s 1; C', s3,2,3⟩
→ ⟨a := 3 ×s 2; y := y −s 1; C', s3,2,3⟩
→ ⟨a := 6; y := y −s 1; C', s3,2,3⟩
→ ⟨skip; y := y −s 1; C', s3,2,6⟩
→ ⟨y := y −s 1; C', s3,2,6⟩
→ ⟨y := 2 −s 1; C', s3,2,6⟩
→ ⟨y := 1; C', s3,2,6⟩
→ ⟨skip; C', s3,1,6⟩
→ ⟨C', s3,1,6⟩
→ ⟨if y >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,1,6⟩
→ ⟨if 1 >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,1,6⟩
→ ⟨if true then (a := a ×s y; y := y −s 1; C') else skip, s3,1,6⟩
→ ⟨a := a ×s y; y := y −s 1; C', s3,1,6⟩
→ ⟨a := 6 ×s y; y := y −s 1; C', s3,1,6⟩
→ ⟨a := 6 ×s 1; y := y −s 1; C', s3,1,6⟩
→ ⟨a := 6; y := y −s 1; C', s3,1,6⟩
→ ⟨skip; y := y −s 1; C', s3,1,6⟩
→ ⟨y := y −s 1; C', s3,1,6⟩
→ ⟨y := 1 −s 1; C', s3,1,6⟩
→ ⟨y := 0; C', s3,1,6⟩
→ ⟨skip; C', s3,0,6⟩
→ ⟨C', s3,0,6⟩
→ ⟨if y >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,0,6⟩
→ ⟨if 0 >s 0 then (a := a ×s y; y := y −s 1; C') else skip, s3,0,6⟩
→ ⟨if false then (a := a ×s y; y := y −s 1; C') else skip, s3,0,6⟩
→ ⟨skip, s3,0,6⟩

```

As you can see, this kind of calculation is horrible to do by hand. It can, however, be automated to give a simple interpreter for the language, based directly on the semantics. It is also formal and precise, with no argument about what should happen at any given point. Finally, it did compute the right answer!

1.4.4 Some Facts

Recall that the small-step semantics for expressions satisfies determinacy, confluence and normalisation, slide 10. Determinacy and confluence hold for the execution relation $\rightarrow_c$ for While, but normalisation does not hold.

Determinacy and Confluence for $\rightarrow_c$

The execution relation $\rightarrow_c$ for While is **deterministic**: that is, for all C, s, C_1, s_1, C_2, s_2 ,

if $\langle C, s \rangle \rightarrow_c \langle C_1, s_1 \rangle$ and $\langle C, s \rangle \rightarrow_c \langle C_2, s_2 \rangle$ then $\langle C_1, s_1 \rangle = \langle C_2, s_2 \rangle$.

Thus, the relation $\rightarrow_c$ is **confluent**: that is, for all C, s, C_1, s_1, C_2 ,

if $\langle C, s \rangle \rightarrow_c^* \langle C_1, s_1 \rangle$ and $\langle C, s \rangle \rightarrow_c^* \langle C_2, s_2 \rangle$ then there exists $\langle C', s' \rangle$ such that $\langle C_1, s_1 \rangle \rightarrow_c^* \langle C', s' \rangle$ and $\langle C_2, s_2 \rangle \rightarrow_c^* \langle C', s' \rangle$

Analogous results hold for the relations $\rightarrow_e$ and $\rightarrow_b$.

Slide 27

However, normalisation does not hold, as illustrated in slide 24. It is possible for a computation to be non-terminating: that is, a computation to result in an infinite loop. Let us prove that `(while true do skip)` never reaches a result, using the fact that the small-step semantics of While is deterministic.

Answer Configurations

Together with the definition of an operational semantics, one needs to define the **answer configurations**. Answer configurations are those configurations which are in normal form, and which describe the answer of some computation.

The answer configurations of $\rightarrow_e$ are $\langle n, s \rangle$, where $n \in \mathbb{N}$.

The answer configurations of $\rightarrow_b$ are $\langle bv, s \rangle$, where $bv \in \{\text{true}, \text{false}\}$.

The answer configurations of $\rightarrow_c$ are $\langle \text{skip}, s \rangle$.

Slide 28

Slide 29

Question: Are all normal form configurations answer configurations?

Stuck Configurations

The configurations in normal form which are not answer configurations, are called **stuck configurations**.

Slide 30

For example, $\langle y, (x \mapsto 3) \rangle$ is a normal expression configuration, since y cannot be evaluated in the state $(x \mapsto 3)$. This configuration is stuck.

Another example is the stuck command configuration $\langle x := y +_s 7; y := y -_s 1, (x \mapsto 3) \rangle$.

A further example is the stuck boolean configuration $\langle 5 <_s y, (x \mapsto 2) \rangle$.

Note that $\langle x < y, (x \mapsto 2) \rangle$ is not stuck, but reduces to a stuck state.

Normalisation

The evaluation relations $\rightarrow_e$ and $\rightarrow_b$ are normalising.

The execution relation $\rightarrow_c$ is not normalising.

Specifically, we can have infinite loops. For example, the program $(\text{while true do skip})$ loops forever.

Let s be any state. We have the execution path

$\langle \text{while true do skip}, s \rangle \rightarrow_c^3 \langle \text{while true do skip}, s \rangle$

where $\rightarrow_c^3$ means three steps. Hence, we have an infinite execution path.

Slide 31

Theorem For any state s , there is no s' such that

$$\langle \text{while true do skip}, s \rangle \rightarrow_c^* \langle \text{skip}, s' \rangle$$

Proof Let us first calculate a few steps of the evaluation of this program:

$$\begin{aligned} & \langle \text{while true do skip}, s \rangle \\ \rightarrow_c & \langle \text{if true then (skip; while true do skip) else skip}, s \rangle \\ \rightarrow_c & \langle \text{skip; while true do skip}, s \rangle \\ \rightarrow_c & \langle \text{while true do skip}, s \rangle \end{aligned}$$

As you can see, it seems unlikely that this will ever get anywhere. But we need to prove this! Suppose to the contrary that it is possible for

$$\langle \text{while true do skip}, s \rangle \rightarrow_c^* \langle \text{skip}, s' \rangle,$$

and let n be the number of steps taken for this evaluation. Note that, since the semantics is deterministic, this number n is well-defined.

Again, determinacy tells us that the first three steps of the evaluation must be the steps we calculated above, and then the remaining $n - 3$ steps of the evaluation show that

$$\langle \text{while true do skip}, s \rangle \rightarrow_c^* \langle \text{skip}, s' \rangle,$$

which is not possible, since this takes n steps! This is a contradiction, so we deduce that no such evaluation exists. ■

This result demonstrates that inherent in our language is the fact that some computations do not yield final answers. It fits our intuition about how programs work!

1.4.5 Discussion: Side-effects and Evaluation Order

Something worth noticing about our language is that the only phrases which affect the state directly are the assignment commands. Assignments are always found inside commands rather than inside expressions or booleans. Furthermore, commands are strictly sequenced by the `;` operator. This means that there is never any confusion about what should be in the state at a given time, and none of the decisions we have made about the order of evaluation affect the final answer configurations.

In more sophisticated languages, this situation can be compromised. For example, commands can often creep into the language of expressions via constructs like a `do...return` expression. For example, we think of

$$\text{do } x := x +_s 1 \text{ return } x$$

as an expression because it returns a numerical result. However, it does have a side-effect on the state. We can also write composite expressions such as

$$(\text{do } x := x +_s 1 \text{ return } x) +_s (\text{do } x := x \times_s 2 \text{ return } x)$$

It is now vital that we pay close attention to the semantics of addition. Does $+_s$ evaluate its argument left-to-right or right-to-left?

Exercise Write down the sets of rules corresponding to each evaluation strategy for $+_s$, and evaluate the above in the state $(x \mapsto 0)$ under each set of rules.

Side-effecting expressions

If we allow expressions like

$$\text{do } x := x +_s 1 \text{ return } x$$

then the result of evaluating

$$(\text{do } x := x +_s 1 \text{ return } x) +_s (\text{do } x := x \times_s 2 \text{ return } x)$$

depends on the evaluation order.

Slide 32

Strictness In the case of addition, the only reasonable choices for evaluation are left-to-right or right-to-left, although other choices do exist, such as evaluating both arguments twice! In any case, it is clear that both arguments must be evaluated at least once before the result of the addition can be calculated. Sometimes, this is not the case. For example, the logical 'and' operator, written $\&_s$ in our syntax, when applied to *false* and any other boolean, must return *false*. It is therefore possible to write a semantics for $\&_s$ as on the previous slide. In this case, the programmer really must know what the semantics is. For example, in any state

$$\text{false} \&_s (\text{do while true do skip; return(true)}) \rightarrow \text{false}$$

while (do while true do skip; return(true)) $\&_s$ false gets into an infinite loop.

Slide 33

Short-circuit Semantics of $\&_s$

$$\frac{B_1 \rightarrow B'_1}{B_1 \&_s B_2 \rightarrow B'_1 \&_s B_2}$$

$$\frac{}{false \&_s B_2 \rightarrow false}$$

$$\frac{}{true \&_s B_2 \rightarrow B_2}$$

Slide 34

Strictness

An operation is called **strict in one of its arguments** if it always needs to evaluate that argument.

Addition is strict in both arguments.

The semantics of $\&_s$ given in slide 33 makes $\&_s$ a **left-strict** operator. It is **non-strict** in its right argument.

Procedure and Method Calls *Though we will not formally consider languages with procedures or method calls, we can informally apply the ideas given in this course to obtain a deeper understanding of such languages. The issues of strictness and evaluation order crop up again and again. For example, in a method like*

```
void aMethod(int x){
    return;
}
```

the argument x is never used. So in a call such as

```
aMethod(do y := y +s 1 return y)
```

do we need to evaluate the argument? Clearly, the outcome of a program containing a call like this will depend on the semantic decision we make here. In a method call such as

```
anotherMethod(exp1, exp2)
```

in what order should we evaluate the arguments, if they are used? If an argument is used twice in the body of the method, should it be evaluated twice? There are plenty of different semantic decisions to be made here. Here are some popular choices.

- *Evaluate all the arguments, left to right, and use the result of this evaluation each time the argument is used. This is called call-by-value, and is roughly what Java and ML do.*
- *Replace each use of an argument in the method body by the text of the actual parameter the programmer has supplied, so that each time the argument is called, it is re-evaluated. This is called call-by-name, and is what Algol 60 did (does?).*
- *Evaluate an argument only when it is actually used in the body of the method, but then remember the result so that if the argument is used again, it is not re-evaluated. This is called call-by-need, and is what Haskell does.*

Exercise *Write the code of a method `myMethod(exp1, exp2)` and a particular call to this method, such that the three evaluation strategies above all give different results.*

The purpose of this discussion is to alert you to the fact that there may be several reasonable, but nonetheless crucially different, choices to be made about the semantics of various language constructs. One role of a formal study of semantics is to help discover where such choices and ambiguities lie, and to resolve them in a fixed, clear and well-documented way.

Summary

- Syntax of *SimpleExp*, *Exp*, *Bool* and *Com*.
- Big-step operational semantics: $\Downarrow \subseteq \text{SimpleExp} \times \mathbb{N}$.
Small-step operational semantics: $\rightarrow \subseteq \text{SimpleExp} \times \text{SimpleExp}$.
- The notion of *State* to describe imperative languages
- Configurations: $\text{Exp} \times \text{State}$, $\text{Bool} \times \text{State}$, $\text{Com} \times \text{State}$.
and small/big step semantics:
 $\rightarrow_e \subseteq (\text{Exp} \times \text{State}) \times (\text{Exp} \times \text{State})$, $\rightarrow_c \subseteq (\text{Com} \times \text{State}) \times (\text{Com} \times \text{State})$
 $\Downarrow_e \subseteq (\text{Exp} \times \text{State}) \times (\mathbb{N} \times \text{State})$, $\Downarrow_c \subseteq (\text{Com} \times \text{State}) \times \text{State}$.
- Total, deterministic, confluent, (unique) normal form, weak/strong normalisation.
Soundness/Completeness of semantics wrt to one another. Stuck configurations. Strictness.
- Sometimes we need to design new configurations (e.g. bean jars.).
- Inference rules can describe many more computations (e.g. beans, heap, registers, types), or inductively defined sets (e.g. syntax).